Camera-Based Parking Slot Occupancy Detection on an ESP32-S3 Using an INT8 TinyML CNN

Druva Teja *CB.SC.U4CSE23317* Sai Jeevan *CB.SC.U4CSE23124* Manish Sakamuri *CB.SC.U4CSE23125* Tejeswara D *CB.SC.U4CSE23614*

Group 79, Department of Computer Science and Engineering, Amrita School of Computing, Amrita Vishwa Vidyapeetham, Coimbatore, India. Project guide: Dr. R. Karthi

***Abstract*—Most smart-parking prototypes either wire a sensor to every slot or push camera frames to something much bigger than a microcontroller. We wanted to see how far a single low-cost camera node could go on its own. This paper describes a parking-slot occupancy classifier that runs entirely on an ESP32-S3 with an OV3660 camera. A 25,730-parameter convolutional network, trained on the public CNRPark+EXT patches and quantized to INT8, shrinks to 33.24 KB and executes under TensorFlow Lite Micro. On the full CNRPark+EXT test split of 31,825 patches the INT8 model reaches 98.01% accuracy (Keras float model: 98.07%), and the board reports a prediction through its own Wi-Fi dashboard in about 497 ms per image. We also test how the model copes with small camera shifts, rotation and lighting changes, since a pole-mounted camera will not stay perfectly still; rotation turns out to be its weak spot. We are equally open about what does not work yet: the prototype classifies a whole frame rather than individual slots, it was not trained on our own campus lot, and it can mistake people for cars. We describe those gaps and the slot-allocation and LoRa back end we plan to build on top.**

***Keywords*—smart parking, TinyML, ESP32-S3, INT8 quantization, convolutional neural network, edge computing, occupancy detection**

I. INTRODUCTION

Finding a free space in a busy lot wastes time and fuel, and the queue that builds at the entrance makes the problem worse for everyone. Many campuses and malls already have the physical layout for a smarter system; what they lack is a cheap way to know which slots are actually free.

The usual answers each have a cost. Per-slot ultrasonic or infrared sensors are accurate but need wiring and maintenance at every bay. Camera systems need far fewer devices, but published designs typically lean on a Jetson-class board or a server running a detector such as YOLO, and cloud-hosted versions stop working when the network does. Our goal was narrower and more practical: put the decision *on the camera node itself*, using an inexpensive off-the-shelf microcontroller board, and see how accurate it can be.

This paper makes three contributions:

1. A complete, reproducible pipeline from a Keras CNN to an INT8 TensorFlow Lite Micro model running on an ESP32-S3, including the camera preprocessing that has to be redone by hand on the device.
2. An evaluation on all 31,825 CNRPark+EXT test patches that compares the float and INT8 models, breaks accuracy down by weather and camera viewpoint, and measures sensitivity to small camera shifts and brightness changes.
3. A plain account of the prototype's limits, together with the allocation and wrong-parking logic that we have designed but not yet implemented.

All code, the model, the sample outputs and the demo videos are in the project repository (Section VII).

II. RELATED WORK

Sensor-based systems. A common design reads one infrared or ultrasonic sensor per slot with a microcontroller and forwards the result to a cloud service. A GIEST 2024 prototype combines IR sensors, an ATmega2560 and an ESP32 that reports to the Blynk cloud, and adds LED strips for indoor guidance [1]. Lubis et al. pair ultrasonic sensors and ESP32 boards with Firebase and then use Faster R-CNN with EasyOCR to check that a reservation matches the plate and slot marker; they report 86% detection accuracy and a 2.7 s average inference-plus-extraction time [2]. These systems work, but wiring grows with the number of slots.

Vision on edge hardware. Rajesh et al. monitor a lot with one infrastructure camera, run the detector on an edge computer to avoid latency, and guide drivers to the nearest free spot [3]. Kim et al. run SSD-MobileNetv2 on a low-cost edge device fed by several IP cameras and add a dual-trigger scheme, periodic plus a parking-space-mask trigger, so the model runs only when the scene changes; they report 0.32 s per frame and an F1-score of 0.9848 over a four-month field test [4]. Alymani et al. combine plate recognition at the gate with slot cameras and infrared sensors [5]. Our work sits at the smaller end of this range: a model that fits on a microcontroller rather than an edge computer.

Allocation. Beyond detection, some papers decide *which* slot to give a driver. Thomas and Kovoor use a genetic algorithm to pick a parking region and an autonomous trolley to park the car [6]; Sakib et al. propose an eco-friendly management scheme in which allocation accounts for emissions and driver priority [7]. We take inspiration from these when describing our planned allocation score in Section VI.

Datasets and TinyML. PKLot [8] and CNRPark+EXT [9] are the two public parking-occupancy datasets we know of with per-space labels. We use CNRPark+EXT because its 150×150 patches are already cropped per slot and labelled. For deployment we rely on TensorFlow Lite Micro [10] and on the integer-only quantization scheme of Jacob et al. [11].

TABLE I. HOW THIS WORK DIFFERS FROM THE CLOSEST PUBLISHED SYSTEMS (AS DESCRIBED IN THEIR ABSTRACTS)

Work	Sensing	Compute at the node	Stated limit / difference
GIEST 2024 [1]	IR per slot	ATmega2560 + ESP32	Wired sensor per slot; no vision
Lubis et al. [2]	Ultrasonic + phone photos	ESP32 + server-side Faster R-CNN, OCR	86% detection, 2.7 s; needs uploaded photos
Rajesh et al. [3]	One infrastructure camera	Edge computer, YOLO-class detector	Far heavier than a microcontroller
Kim et al. [4]	IP cameras	Edge device, SSD-MobileNetv2	0.32 s/frame, F1 0.9848 (field)
This work	OV3660 on the board	**ESP32-S3, 33 KB INT8 CNN**	~497 ms/image; CNR-EXT only, no campus field data yet

III. SYSTEM DESIGN

A. Hardware

The node is an ESP32-S3-WROOM-1 N16R8 development board (dual-core 240 MHz, 8 MB PSRAM, 16 MB flash) with an OV3660 camera module attached directly to the board. We first bought a generic ESP32-S3 DevKit and discovered that it has no flat-cable camera connector, so the ESP32-CAM sensor could not be reused; switching to a board with the camera built in fixed this. The internal SRAM is only about 512 KB, which matters later.

B. Network architecture

Fig. 1 shows the CNN. It has three 3×3 convolution blocks with 16, 32 and 64 filters, two max-pooling layers, global average pooling in place of a flatten layer, a 32-unit dense layer, dropout of 0.3 and a two-way softmax. The total is 25,730 parameters, about 100.5 KB as 32-bit floats. Global average pooling is the key size saver: a flatten layer after the last convolution would have multiplied the dense-layer weights many times over.

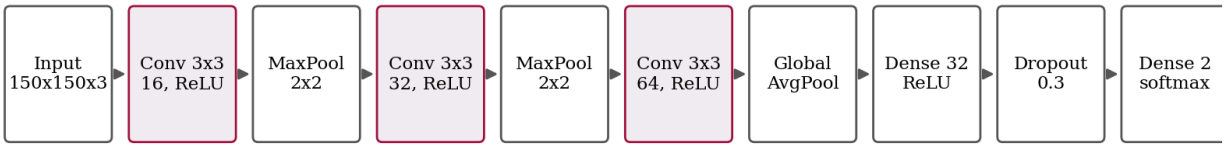


Fig. 1. The classifier. Convolution layers are outlined in maroon.

C. Data and training

CNRPark+EXT provides 150×150 RGB patches, one parking space per patch, labelled 0 (empty) or 1 (occupied), with predefined train, validation and test lists. The lists contain 94,493, 18,647 and 31,825 images respectively. To keep early experiments quick we trained on a class-balanced random subset of 20,000 training images (10,000 per class, seed 42) and evaluated on the untouched validation and test lists. Training used Adam (learning rate 0.001), batch size 32, up to 20 epochs with early stopping on validation loss (patience 4), and light augmentation—rotation up to ±0.03 turn (about ±11°), translation and zoom up to 3% and brightness change up to 10%—chosen to mimic a camera that has drifted slightly. Pixels are scaled to [0, 1].

The Keras model reached 98.11% on the test list in our training run; Section IV re-measures it (slightly differently, because of image-loading details). A 96×96 variant trained the same way reached 98.25%, but we deployed the 150×150 model because its input matches the stored patches and the memory cost was acceptable.

D. INT8 quantization

We used post-training full-integer quantization with the TensorFlow Lite converter. A representative set of 500 randomly chosen patches calibrated activation ranges; input and output tensors are INT8. A pixel x in [0, 1] is stored as $q = round(x/s) + z$ with the input scale and zero point read from the model, and the two outputs are converted back with $x = (q - z)s$ before taking the larger. The result is a 34,040-byte file (33.24 KB), about a third of the float size. The calibration set was drawn from the full patch folder without excluding test images; it is only used to set ranges, not to train, but we note it for completeness.

E. On-device pipeline

The model is embedded in the firmware as a C array and executed with TensorFlow Lite Micro using a handful of operators (convolution, pooling, mean, fully connected, softmax, reshape, quantize and dequantize). Fig. 2 shows what happens at runtime. This is where most of our debugging time went. The camera delivers RGB565 frames at 320×240; the training images were 8-bit RGB, so each frame is resized to 150×150 and every pixel expanded to RGB888 by hand before quantizing. The tensor arena is 600 KB and is placed in PSRAM; our first attempt in internal RAM failed with a request for 469,136 bytes against about 151,000 available. An old community TensorFlow library also failed to compile against the newer API, so we switched to Espressif's own TFLite Micro port.



Fig. 2. Runtime pipeline on the ESP32-S3.

For the demo the board captures and classifies one frame every 30 s and serves the result from its own Wi-Fi access point at 192.168.4.1: the latest image, the label, the two class probabilities, and capture and inference times.

IV. EXPERIMENTS AND RESULTS

A. Float versus INT8

We ran the float Keras model and the INT8 TFLite model (with the same quantize-run-dequantize steps as the firmware) over all 31,825 test patches. Table II shows that quantization cost very little: the INT8 model scores 98.01% against 98.07% for float, and the two models agree on 99.66% of patches. Precision, recall and F1 for the occupied class are 98.52%, 98.02% and 98.27% (Fig. 3).

TABLE II. TEST-SPLIT RESULTS (31,825 PATCHES)

Model	Size	Accuracy
Keras float32	~100.5 KB	98.07%
TFLite INT8	33.24 KB	98.01%
Float/INT8 agreement	–	99.66%

Errors of the INT8 model: 270 empty patches called occupied (1.99% of empty) and 362 occupied patches called empty (1.98% of occupied).

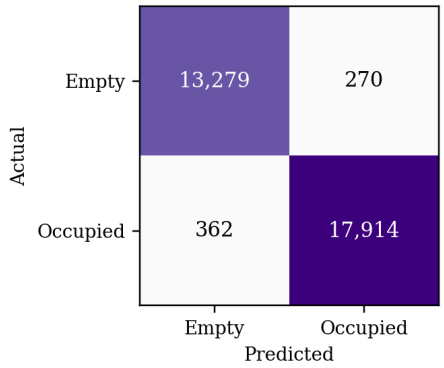


Fig. 3. Confusion matrix, INT8 model, full test split.

B. Weather and viewpoint

CNRPark+EXT contains sunny, overcast and rainy days and nine camera viewpoints. Fig. 4 splits the INT8 accuracy by both. Accuracy is highest on overcast days (99.14%) and lowest on rainy days (97.14%). Across viewpoints it ranges from 95.57% (camera1) to 99.15% (camera5), so a few angles are clearly harder than others. This is still the same dataset family the model was trained on, so it says little about a completely new lot.

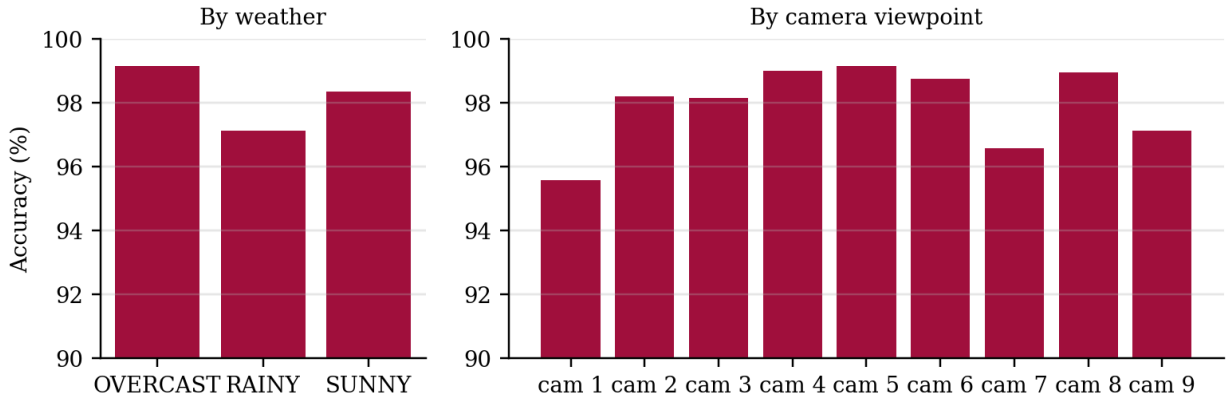


Fig. 4. INT8 accuracy by weather condition and camera viewpoint on the test split.

C. Robustness to camera movement and lighting

Our guide asked that the system tolerate a camera that moves slightly with wind or maintenance. We could not physically bump a camera hundreds of times, so we applied synthetic perturbations to a fixed random subset of 4,000 test patches and re-ran the INT8 model: a 10-pixel horizontal or vertical shift, a 5° rotation, a 10% zoom-in crop, and brightness scaled by 0.6 and 1.4. Table III gives the results. Shifts, zoom and brightness changes cost between 0.2 and 1.9 points. The 5° rotation was the exception: accuracy fell from 98.28% to 79.05%. Part of that is probably an artefact of our test, because rotating a patch leaves black corners that the model never saw during training (Keras augmentation fills them by reflection), but we cannot rule out a genuine weakness to tilt. We treat it as an open problem.

TABLE III. INT8 ACCURACY UNDER SYNTHETIC CAMERA AND LIGHTING CHANGES (4,000 PATCHES)

Perturbation	Accuracy	Change
original	98.28%	–
shift x 10px	98.10%	-0.18 pt
shift y 10px	96.33%	-1.95 pt
rotate 5deg	79.05%	-19.23 pt
zoom in 10pct	98.00%	-0.28 pt
brightness x0.6	96.35%	-1.93 pt
brightness x1.4	96.63%	-1.65 pt

D. On-device behaviour

On the ESP32-S3 a classification takes about 497 ms and is nearly constant from frame to frame, because the network does the same arithmetic every time. The first camera capture after boot takes about 300 ms while the sensor settles, and later captures about 7 ms. The model plus runtime fit comfortably: one of our test sketches used 29% of program storage and 53% of dynamic memory with a 150 KB arena in internal RAM (it later moved to PSRAM). Compared with the 0.32 s per frame that Kim et al. report on a larger edge device [4], half a second on a microcontroller is acceptable for a status that changes over minutes, and the dashboard refreshes only every 30 s anyway.



Fig. 5. Predictions of the deployed INT8 model on sample CNRPark+EXT test patches (green titles are correct; the label and probability are shown).

V. DISCUSSION AND LIMITATIONS

We would rather list what is still wrong than have a reader find it.

The numbers are benchmark numbers. 98.01% describes performance on CNRPark+EXT patches. We have not yet collected and labelled a set from our own campus lot, so we cannot claim that accuracy for our deployment. Fig. 5 shows deployed-model predictions on sample patches; campus photos are in the repository but were used only for trial runs.

Whole frame instead of slots. The model learned from patches that each show a single space, but the demo firmware resizes the entire 320×240 frame to 150×150. In practice a person or a bag in view was sometimes labelled occupied. Cropping a fixed region of interest per slot before inference is the obvious fix, and we expect it to matter more than any change to the network. A temporal rule, such as requiring two or three identical predictions in a row, would also steady the output.

Training-data balance. The training subset contains cars and empty ground but no people, animals or luggage, so the model has no reason to call those "empty". Whether they should count as occupied is a policy choice we still need to make and label for.

Power. The dashboard worked when the board was plugged into a laptop but not reliably from a power bank, which points to a supply or brown-out reset problem rather than a software one. We have not isolated it yet.

Synthetic perturbations and tilt. Shifting or rotating a patch is not the same as moving a real camera, which changes perspective and what the patch contains. Table III is a sanity check, not a substitute for field testing. The sharp drop under rotation is the result that worries us most, because wind-induced tilt is exactly the case our guide described. Retraining with rotated inputs that keep their borders, and testing on real tilted captures, are priorities.

VI. PLANNED SLOT ALLOCATION AND BACK END

The occupancy node is the foundation; the rest of the system is designed but not yet implemented, and we label it that way deliberately. Fig. 6 shows the intended architecture: several camera nodes send compact occupancy vectors over LoRa to a Raspberry Pi 3A+ that stores state in SQLite, generates a dynamic entry QR code, assigns a slot, sends the driver a web page and SMS, and syncs to a cloud database when a connection is available.

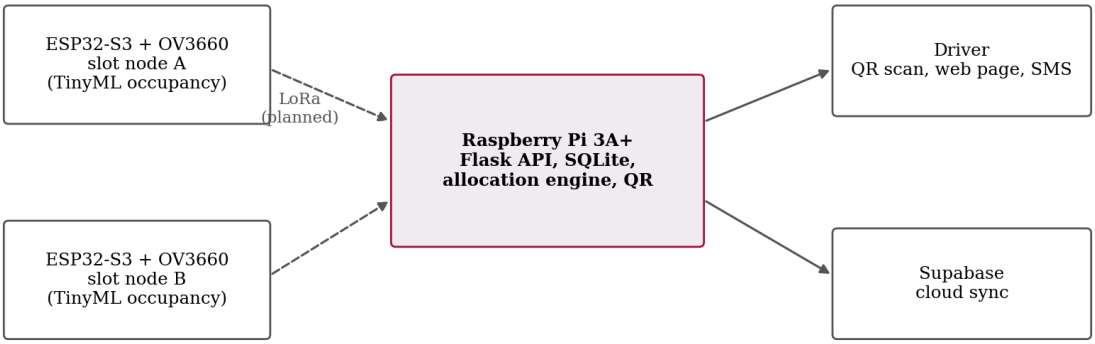


Fig. 6. Planned system architecture. Only the camera node (left) is implemented.

For each free slot s the allocation engine will compute

$$Score(s) = w_1 D_e(s) + w_2 D_l(s) + w_3(\rho_r + \alpha T_r)$$

and choose the slot with the lowest score. Here D_e and D_l are normalised distances to the entrance and to the nearest lift, ρ_r is the occupancy ratio of the slot's row, T_r counts recent allocations in that row, and the weights sum to 1. The last term stops consecutive cars being sent to the same row, which is what causes congestion inside a lot. To catch drivers who park in the wrong place we plan a time-window check: when a slot is assigned the system expects it to become occupied within a few minutes; if a different, unassigned slot fills instead, the entry is matched to it by timestamp and the driver is asked by SMS to confirm. These ideas are not yet evaluated, and we make no accuracy claims about them.

VII. REPRODUCIBILITY

The repository contains the firmware sketches, training and conversion scripts, the INT8 model, the C array, 60 sample patches, the demo videos, and two scripts: `demo/run_demo.py` regenerates the sample outputs and the full-test metrics, and `demo/analysis.py` regenerates the float-versus-INT8, weather, camera and robustness tables in this paper. The repository is at github.com/ballsycodes/Automated-Slot-Allocated-Smart-Parking-System.

VIII. CONCLUSION

A 33 KB INT8 convolutional network is enough to classify parking-slot patches with about 98.01% accuracy on a public benchmark, and an ESP32-S3 can run it in roughly half a second without any server. Quantization cost almost nothing in accuracy, and the model tolerated small synthetic shifts and brightness changes, although a 5° rotation cut accuracy sharply. The distance between that and a working parking system is still real: per-slot cropping, retraining on our own lot, resolving the power issue, and building the LoRa, allocation and notification layers. Those are our next steps, in that order.

ACKNOWLEDGMENT

We thank our guide, Dr. R. Karthi, for pushing us to ask what happens when the camera moves, and the CNRPark+EXT authors for releasing their data.

REFERENCES

1. D. Bhatnagar, V. Soni, and P. Agrawal, "IoT-enabled smart car parking system with route navigation," in *Proc. 2024 IEEE 1st Int. Conf. Green Industrial Electronics and Sustainable Technologies (GIEST)*, Oct. 2024, pp. 1–5, doi: 10.1109/GIEST62955.2024.10959768.
2. F. Lubis, E. B. Nababan, P. I. Nainggolan, M. F. S. Putra, R. Sulaiman, R. F. Rahmat, and A. L. T. Pangaribuan, "Integrated smart parking system using Internet of Things (IoT) and digital image processing with Faster R-CNN and OCR," in *Proc. 2024 8th Int. Conf. Electrical, Telecommunication and Computer Engineering*

(ELTICOM), 2024, pp. 342–348, doi: 10.1109/ELTICOM64085.2024.10864954.

3. P. Rajesh, D. Kallakuri, S. Chagarlamudi, S. M. Vissavajhula, J. Meng, and J. Zhao, "Intelligent parking guidance system using computer vision, IoT, and edge computing," *IEEE Open J. Intell. Transp. Syst.*, vol. 6, pp. 1185–1199, 2025, doi: 10.1109/OJITS.2025.3605318.
4. J. Kim, I. Jeong, J. Jung, and J. Cho, "Resource-efficient design and implementation of real-time parking monitoring system with edge device," *Sensors*, vol. 25, no. 7, p. 2181, 2025, doi: 10.3390/s25072181.
5. M. Alymani, L. A. Almoqhem, D. A. Alabdulwahab, A. A. Alghamdi, H. Alshahrani, and K. Raza, "Enabling smart parking for smart cities using Internet of Things (IoT) and machine learning," *PeerJ Comput. Sci.*, vol. 11, p. e2544, 2025, doi: 10.7717/peerj-cs.2544.
6. D. Thomas and B. C. Kovoor, "A genetic algorithm approach to autonomous smart vehicle parking system," *Procedia Comput. Sci.*, vol. 125, pp. 68–76, 2018, doi: 10.1016/j.procs.2017.12.011.
7. N. Sakib, A. S. M. Bakibillah, S. Susilawati, M. A. S. Kamal, and K. Yamada, "Eco-friendly smart car parking management system with enhanced sustainability," *Sustainability*, vol. 16, no. 10, p. 4145, 2024, doi: 10.3390/su16104145.
8. P. R. L. de Almeida, L. S. Oliveira, A. S. Britto Jr., E. J. Silva Jr., and A. L. Koerich, "PKLot–A robust dataset for parking lot classification," *Expert Syst. Appl.*, vol. 42, no. 11, pp. 4937–4949, 2015.
9. G. Amato, F. Carrara, F. Falchi, C. Gennaro, C. Meghini, and C. Vairo, "Deep learning for decentralized parking lot occupancy detection," *Expert Syst. Appl.*, vol. 72, pp. 327–334, 2017.
10. R. David *et al.*, "TensorFlow Lite Micro: Embedded machine learning for TinyML systems," in *Proc. Machine Learning and Systems (MLSys)*, vol. 3, 2021, pp. 800–811.
11. B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.